Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «КиСБ»

По теме: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4-го курса
Группы АС-65
Осовец М. М.
Проверил:
Хацкевич А. С.

Брест 2026

Часть 1

Цель работы: освоить основные алгоритмы программного генерирования равномерно распределенных псевдослучайных последовательностей.

Задание

1. Согласно варианту реализовать приложение, генерирующие псевдослучайную равномерно распределенную последовательность произвольной длины из заданного алфавита.
2. Подобрать параметры данного генератора таким образом, чтобы период последовательности имел максимальное значение.
3. Используя статистическое тестирование, проверить гипотезу о том, что сгенерированная последовательность действительно имеет равномерное распределение (см. приложение 2).
4. Отчет должен обязательно содержать несколько тестов, отображающих изменение статистических свойств последовательностей в зависимости от параметров генератора.

4.1.2.2. Генератор Эйхенауэра – Лена с обращением.

Псевдослучайная нелинейная конгруэнтная последовательность Эйхенауэра – Лена с обращением определяется следующим нелинейным рекуррентным соотношением:

$$x_{t+1} = \begin{cases} \left(ax_t^{-1} + c\right) \bmod N, & \text{если } x_t \geq 1 \\ c, & \text{если } x_t = 0 \end{cases} \quad \textbf{(3)}$$

где x_t^{-1} – обратный к x_t элемент по модулю N, т.е. $x_t x_t^{-1} \equiv 1 (\bmod N)$; $x_0, a, c \in A$ – параметры генератора.

Выбор параметров осуществляется на основании свойства:

1. Если $N = 2^q$, a, x_0 – нечетны, c — четно, то генератор (3) имеет максимально возможный период $T_{max} = 2^{q-1}$ тогда и только тогда, когда $a \equiv 1 (\bmod 4)$, $c \equiv 2 (\bmod 4)$

Общее начало в коде:

```
import math
```

```
import time
import csv
from collections import Counter
import matplotlib.pyplot as plt

# Лабораторная работа №1, вариант 3
# Часть 1: инверсный генератор Эйхенауэра — Лена, алфавит N=20.
# Часть 2: полиномиальный тест простоты AKS.

N = 20
MODULUS = 1009
A = 2
C = 1
X0 = 1
SEED = 42
```

Первая часть (код):

```
# -------------------- Часть 1 --------------------


def next_state(x, a=A, c=C, modulus=MODULUS):
    """Один шаг инверсного конгруэнтного генератора."""
    inverse = 0 if x == 0 else pow(x, -1, modulus)
    return (a * inverse + c) % modulus


def generate_sequence(length, a=A, c=C, x0=X0, modulus=MODULUS):
    """Генерирует последовательность символов алфавита {0,...,N-1}."""
    if length < 0:
        raise ValueError("Длина последовательности не может быть отрицатель-
ной")
    x = x0 % modulus
    result = []
    for _ in range(length):
        result.append(x % N)
        x = next_state(x, a, c, modulus)
```

```
    return result


def generate_states(length, a=A, c=C, x0=X0, modulus=MODULUS):
    """Возвращает внутренние состояния; они же используются для кандида-
тов."""
    x = x0 % modulus
    states = []
    for _ in range(length):
        states.append(x)
        x = next_state(x, a, c, modulus)
    return states


def find_period(a=A, c=C, x0=X0, modulus=MODULUS):
    """Точный период внутреннего состояния от заданного x0."""
    seen = {}
    x = x0 % modulus
    step = 0
    while x not in seen:
        seen[x] = step
        x = next_state(x, a, c, modulus)
        step += 1
    return step - seen[x]


def search_parameters(modulus=MODULUS, max_a=30, max_c=30, x0=X0):
    """Ищет лучший период среди ограниченного набора a и c."""
    best = []
    best_period = -1
    for a in range(1, max_a + 1):
        for c in range(1, max_c + 1):
            period = find_period(a, c, x0, modulus)
            if period > best_period:
                best_period = period
                best = [(a, c, period)]
```

```
            elif period == best_period:
                best.append((a, c, period))
    return best_period, best


def frequency_test(sequence):
    total = len(sequence)
    if total == 0:
        raise ValueError("Последовательность не должна быть пустой")
    counts = Counter(sequence)
    expected = total / N
    chi2 = sum((counts.get(i, 0) - expected) ** 2 / expected for i in
range(N))
    return counts, chi2


def run_part1():
    print("=" * 62)
    print("ЧАСТЬ 1. ИНВЕРСНЫЙ ГЕНЕРАТОР ЭЙХЕНАУЭРА — ЛЕНА")
    print("=" * 62)
    length = 10000
    sequence = generate_sequence(length)
    period = find_period()
    counts, chi2 = frequency_test(sequence)

    print(f"Алфавит N={N}; модуль={MODULUS}; a={A}; c={C}; x0={X0}")
    print(f"Длина последовательности: {length}")
    print(f"Период внутреннего состояния: {period}")
    print(f"Ожидаемая частота каждого символа: {length / N:.2f}")
    print(f"Статистика хи-квадрат: {chi2:.4f}")
    print("Символ | Количество | Частота, %")
    for symbol in range(N):
        count = counts.get(symbol, 0)
        print(f"{symbol:6} | {count:10} | {count / length * 100:9.2f}")

    with open("frequency_table.csv", "w", newline="", encoding="utf-8") as f:
```

```
    writer = csv.writer(f)
    writer.writerow(["symbol", "count", "frequency_percent"])
    for symbol in range(N):
        count = counts.get(symbol, 0)
        writer.writerow([symbol, count, count / length * 100])

plt.figure(figsize=(10, 5))
plt.bar(range(N), [counts.get(i, 0) for i in range(N)])
plt.axhline(length / N, linestyle="--", label="Ожидаемая частота")
plt.xlabel("Символ алфавита")
plt.ylabel("Количество появлений")
plt.title("Частотное распределение символов (N=20)")
plt.legend()
plt.tight_layout()
plt.savefig("frequencies.png", dpi=150)
plt.close()

# Сравнение периодов для разных параметров
variants = [(2, 1, 1), (3, 1, 1), (5, 7, 1), (7, 11, 1)]
print("\nСравнение периодов:")
labels, periods = [], []
for a, c, x0 in variants:
    p = find_period(a, c, x0)
    labels.append(f"a={a}, c={c}")
    periods.append(p)
    print(f"a={a}, c={c}, x0={x0}: период={p}")

plt.figure(figsize=(8, 5))
plt.bar(labels, periods)
plt.ylabel("Период внутреннего состояния")
plt.title("Сравнение периодов при разных параметрах")
plt.tight_layout()
plt.savefig("periods.png", dpi=150)
plt.close()

print("\nПоиск лучшего периода для 1 <= a,c <= 30...")
```

```
    best_period, best_params = search_parameters()
    print(f"Лучший найденный период в этом диапазоне: {best_period}")
    print("Параметры (a, c, период):", best_params[:10])
    print(
        "Примечание: это максимум только среди проверенных параметров, не доказательство глобального максимума."
    )
    return sequence
```

Часть 2

Цель работы: освоить основные программные методы тестирования чисел на простоту.

Задание

1. Реализовать приложение, позволяющее генерировать простое число по следующей схеме (с использованием, например, теста Рабина – Миллера):
    1. Сгенерируйте случайное p-битовое число n.
    2. Установите старший и младший биты равными *1*. (Старший бит гарантирует требуемую длину простого числа, а младший бит обеспечивает его нечетность.)
    3. Убедитесь, что n не делится на небольшие простые числа: 3, 5, 7, 11, и т.д. Во многих реализациях проверяется делимость n на все простые числа, меньшие 256. Наиболее эффективной является проверка на делимость для всех простых чисел, меньших 2000.

Все простые числа, не превосходящие *256* перечислены ниже:
2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,
83,89,97,101,103,107,109,113,127,131,137,139,149,151,157,
163,167,173,179,181,191,193,197,199,211,223,227,229,233,239,
241,251.

4. Выполните один тест Рабина – Миллера проверки простоты числа для некоторого случайного a. Если n проходит тест, сгенерируйте другое случайное a и повторите проверку. Выбирайте небольшие значения a для ускорения вычислений. Выполните пять тестов. (Одного может показаться достаточным, но выполните пять.) Если n не проходит одной из проверок, сгенерируйте другое n и попробуйте снова.

Можно не генерировать n случайным образом каждый раз, но последовательно перебирать числа, начиная со случайно выбранного до тех пор, пока не будет найдено простое число.

Этап (3) не является обязательным, но это хорошая идея. Проверка, что случайное нечетное n не делится на *3*, *5* и *7* отсекает *54* процента нечетных чисел еще до этапа (4). Проверка делимости на все простые числа, меньшие *100*, убирает *76* процентов нечетных чисел, проверка делимости на все простые числа, меньшие *256*, убирает *80* процентов нечетных чисел. В общем случае, доля нечетных кандидатов, которые не делятся ни на одно простое число, меньшее z, равна $1.12 / \ln z$.

Вместо теста Рабина – Миллера следует использовать тест, заданный в варианте.

2. В отчете обязательно должны присутствовать тесты, подтверждающие правильность разработанных программ.

6.8. Полиномиальный тест распознавания простоты.

Данный алгоритм основан на следующем критерии простоты.

Теорема 3.

Пусть числа a, n взаимно просты. Тогда n – простое в том и только в том случае, когда выполнено сравнение:

$$(x - a)^n \equiv (x^n - a) \pmod{n} \qquad \textbf{(3)}$$

Приведем сам алгоритм.

Вход: целое $n > 1$.

1. Если число n имеет вид a^b, то ответ «n – составное».
2. $r \leftarrow 2$
3. Цикл пока $r < n$:
4. Если $НОД(r, n) \neq 1$, то ответ «n – составное».

5. Если r – простое, то вычислить q – наибольший простой делитель $r-1$; если $\left(q > 4\sqrt{r}\ log_2 n\right)$ и $n^{\frac{r-1}{q}} \neq 1 (mod\ r)$, то выйти из цикла.
6. $r \leftarrow r+1$.
7. Завершить цикл.
8. Если $r = n$, то ответ «n – составное».
9. Если $n-1 \leq \left[2\sqrt{r}\ log_2 n\right]$, то $\forall a = \overline{r, n-1}$ проверить выполнение условия: $НОД(a, n) = 1$.
10. Если $n-1 > \left[2\sqrt{r}\ log_2 n\right]$, то $a = \overline{1,2\sqrt{r}\ log_2 n}$ проверить выполнение условия: $(x-a)^n = (x^n - a)(mod\ x^r - 1)$.
11. Если на шаге 9-10 нарушились равенства, то ответ «n – составное».
12. Если мы дошли до этого шага, то ответ «n – простое».

Вторая часть (код):

```
# -------------------- Часть 2: AKS --------------------


def is_perfect_power(n):
    if n < 4:
        return False
    max_exp = n.bit_length()
    for exponent in range(2, max_exp + 1):
        low, high = 2, 1 << ((n.bit_length() + exponent - 1) // exponent)
        while low <= high:
            mid = (low + high) // 2
            value = mid**exponent
            if value == n:
                return True
            if value < n:
                low = mid + 1
            else:
                high = mid - 1
    return False


def multiplicative_order(n, r):
    if math.gcd(n, r) != 1:
```

```
        return 0
    value = 1
    for k in range(1, r + 1):
        value = value * n % r
        if value == 1:
            return k
    return 0


def euler_phi(n):
    result = n
    value = n
    p = 2
    while p * p <= value:
        if value % p == 0:
            while value % p == 0:
                value //= p
            result -= result // p
        p += 1
    if value > 1:
        result -= result // value
    return result


def poly_mul(left, right, r, modulus):
    result = [0] * r
    for i, x in enumerate(left):
        if not x:
            continue
        for j, y in enumerate(right):
            if y:
                result[(i + j) % r] = (result[(i + j) % r] + x * y) % modulus
    return result


def poly_pow(base, exponent, r, modulus):
    result = [0] * r
```

```
    result[0] = 1
    base = base[:]
    while exponent:
        if exponent & 1:
            result = poly_mul(result, base, r, modulus)
        base = poly_mul(base, base, r, modulus)
        exponent >>= 1
    return result


def aks_is_prime(n):
    """Детерминированный AKS для учебных чисел; на больших n может быть медленным."""
    if n < 2:
        return False
    small_primes = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37)
    for p in small_primes:
        if n == p:
            return True
        if n % p == 0:
            return False

    if is_perfect_power(n):
        return False

    log_bound = math.ceil(math.log2(n) ** 2)
    r = 2
    while True:
        if math.gcd(n, r) == 1 and multiplicative_order(n, r) > log_bound:
            break
        r += 1

    for a in range(2, min(r, n)):
        d = math.gcd(a, n)
        if 1 < d < n:
            return False
```

```
    if n <= r:
        return True

    bound = int(math.sqrt(euler_phi(r)) * math.log2(n))
    for a in range(1, bound + 1):
        base = [a % n, 1] + [0] * (r - 2)
        left = poly_pow(base, n, r, n)
        right = [0] * r
        right[0] = a % n
        right[n % r] = (right[n % r] + 1) % n
        if left != right:
            return False
    return True


def small_prime_sieve(n, limit=2000):
    """True, если кандидат не делится на простые числа <= limit."""
    if n < 2:
        return False
    for p in range(2, min(limit, math.isqrt(n)) + 1):
        prime = True
        for d in range(2, math.isqrt(p) + 1):
            if p % d == 0:
                prime = False
                break
        if prime:
            if n == p:
                return True
            if n % p == 0:
                return False
    return True


def candidate_from_generator(bits, sequence):
    """Получает биты из внутреннего состояния генератора варианта 3."""
    if bits < 2:
        raise ValueError("bits должно быть >= 2")
```

```
    needed = bits
    bit_string = []
    # Берём младший бит каждого внутреннего состояния.
    # Для учебной демонстрации; это не криптографический генератор.
    for state in sequence:
        bit_string.append(str(state & 1))
        if len(bit_string) >= needed:
            break
    while len(bit_string) < needed:
        # Продолжаем последовательность при необходимости.
        bit_string.append("0")
    n = int("".join(bit_string), 2)
    n |= 1 << (bits - 1)  # старший бит = 1
    n |= 1  # младший бит = 1
    return n


def generate_prime(bits, sequence, max_candidates=10000):
    """Возвращает найденное простое число и счётчики фильтрации."""
    tested = sieve_rejected = aks_rejected = 0
    start = time.perf_counter()
    offset = 0

    while tested < max_candidates:
        # Перестраиваем кандидат из разных участков потока генератора.
        # Для кандидата используем внутренние состояния генератора,
        # а не символы алфавита после операции % N.
        chunk = generate_states(bits, x0=(X0 + offset) % MODULUS)
        offset += bits
        n = candidate_from_generator(bits, chunk)
        tested += 1

        if not small_prime_sieve(n, 2000):
            sieve_rejected += 1
            continue
        if not aks_is_prime(n):
            aks_rejected += 1
```

```
            continue

        elapsed = time.perf_counter() - start
        return n, tested, sieve_rejected, aks_rejected, elapsed

    raise RuntimeError("Не удалось найти простое число: увеличьте
max_candidates")


def run_part2(sequence):
    print("\n" + "=" * 62)
    print("ЧАСТЬ 2. ПОЛИНОМИАЛЬНЫЙ ТЕСТ ПРОСТОТЫ AKS")
    print("=" * 62)
    test_numbers = [2, 3, 17, 25, 97, 121, 257, 561, 1009]
    print("Проверка тестовых чисел:")
    for n in test_numbers:
        print(f"{n:5} -> {'простое' if aks_is_prime(n) else 'составное'}")

    bits = 12
    prime, tested, sieve_rejected, aks_rejected, elapsed = generate_prime(
        bits, sequence
    )
    print(f"\nНайдено простое число ({bits} бит): {prime}")
    print(f"Повторная проверка AKS: {aks_is_prime(prime)}")
    print(f"Всего проверено кандидатов: {tested}")
    print(f"Отсеяно просеиванием: {sieve_rejected}")
    print(f"Отсеяно тестом AKS: {aks_rejected}")
    print(f"Время поиска: {elapsed:.6f} сек.")

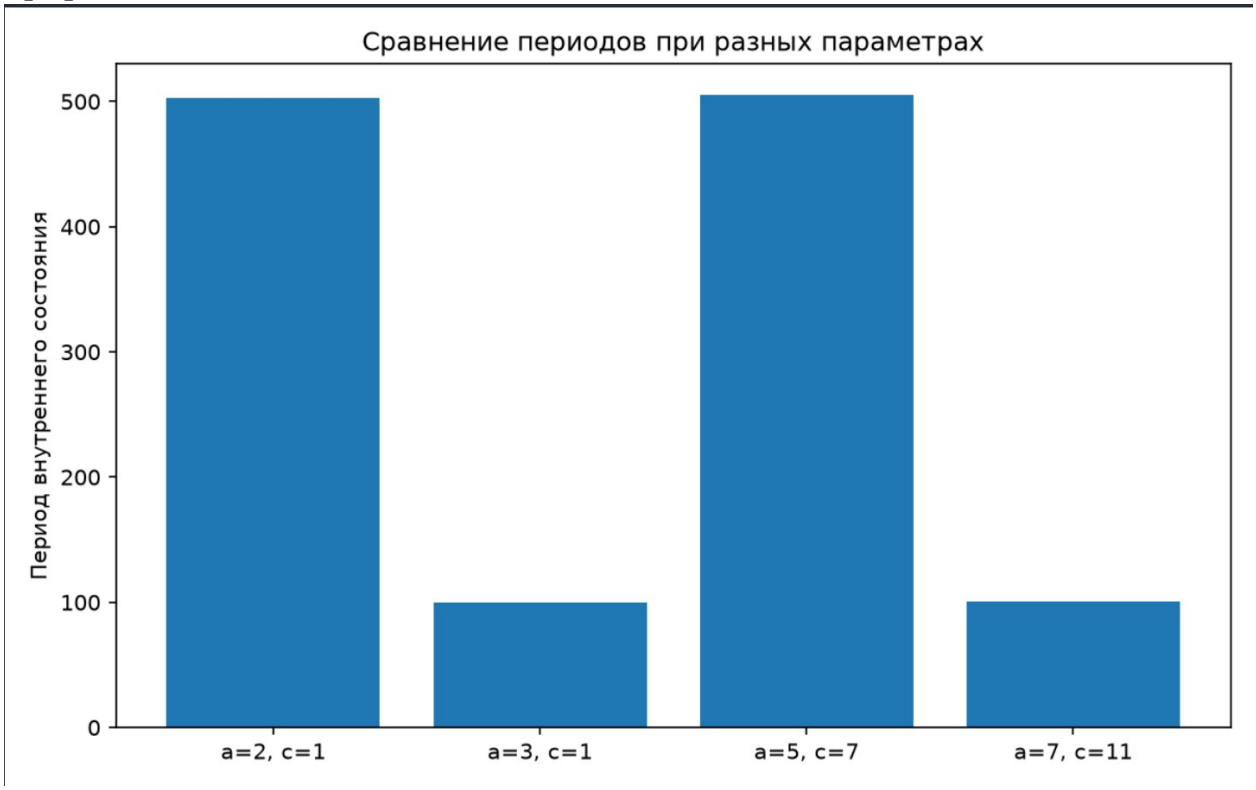
    with open("prime_search_log.csv", "w", newline="", encoding="utf-8") as
f:
        writer = csv.writer(f)
        writer.writerow(
            [
                "bits",
                "prime",
                "candidates_tested",
```

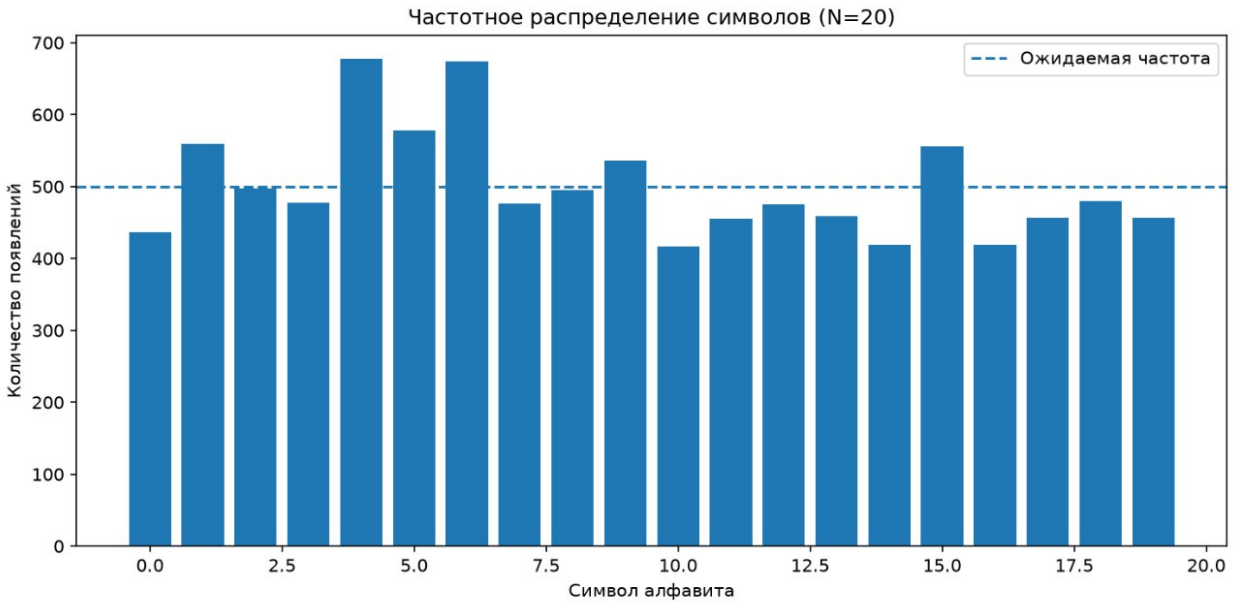
```
                "sieve_rejected",
                "aks_rejected",
                "elapsed_seconds",
            ]
        )
        writer.writerow(
            [bits, prime, tested, sieve_rejected, aks_rejected,
f"{elapsed:.6f}"]
        )
```

Общая концовка в коде:

```
if __name__ == "__main__":
    # Фиксируем воспроизводимость эксперимента.
    print("Лабораторная работа №1. Вариант 3")
    sequence = run_part1()
    run_part2(sequence)
    print(
        "\nФайлы результатов: frequencies.png, periods.png,
frequency_table.csv, prime_search_log.csv"
    )
```

Графики:





CSV-файлы:

```
symbol,count,frequency_percent
0,437,4.37
1,559,5.59
2,497,4.97
3,477,4.77
4,677,6.77
5,578,5.779999999999999
6,674,6.74
7,476,4.760000000000001
8,495,4.95
9,536,5.36
10,417,4.17
11,455,4.55
12,475,4.75
13,459,4.590000000000001
14,419,4.19
15,556,5.56
16,419,4.19
17,457,4.569999999999999
18,480,4.8
19,457,4.569999999999999
```

```
bits,prime,candidates_tested,sieve_rejected,aks_rejected,elapsed_seconds
12,3613,6,5,0,2.895713
```

Вывод: В ходе лабораторной работы были изучены и реализованы алгоритмы генерации псевдослучайных последовательностей, методы их статистического анализа и проверки чисел на простоту. Был исследован период генератора, оценена равномерность распределения полученных значений, а также реализована генерация больших простых чисел с предварительной фильтрацией и проверкой кандидатов. В результате были закреплены практические навыки работы с генераторами псевдослучайных чисел и алгоритмами криптографии.